Lab -10: Experiment No: 10

Aim

To establish a permissioned blockchain network using Hyperledger Fabric, deploy a smart contract (chaincode), and demonstrate asset initialization, creation, retrieval and ownership transfer.

Theory

1. Hyperledger Fabric

Hyperledger Fabric is an open-source blockchain framework designed for enterprise applications. Unlike public blockchain networks, Fabric operates in a permissioned environment where only authenticated and authorized participants can access the network.

It supports secure transactions, distributed ledger maintenance and automated business operations through smart contracts called chaincode.

2. Components of Hyperledger Fabric

1. Peer Nodes: Peers maintain copies of the blockchain ledger and execute chaincode. In this experiment, two peer nodes belonging to separate organizations participate in the network.
2. Ordering Service: The orderer arranges endorsed transactions into blocks and distributes them to the participating peers.
3. Channel: A channel provides a shared communication and ledger environment for selected organizations. The channel created in this experiment is named mychannel.
4. Chaincode: Chaincode represents the business logic of a blockchain application. The asset-transfer-basic JavaScript chaincode is used to manage asset information and ownership.
5. Membership Service Provider (MSP): MSP manages participant identities and determines which organizations and users are authorized to interact with the network.
6. Distributed Ledger: The ledger maintains the current asset records along with the blockchain transaction history.

3. Transaction Processing

Hyperledger Fabric follows the **Execute–Order–Validate** transaction model.

Initially, the transaction proposal is executed by endorsing peers. After obtaining the required endorsements, the transaction is submitted to the ordering service. The ordered transaction is then validated and committed to the blockchain ledger.

This approach helps maintain consistency and integrity across participating organizations.

Software and Tools Required

- Hyperledger Fabric 2.5.x
- Docker Desktop
- Ubuntu through Windows Subsystem for Linux (WSL2)
- Fabric Samples and Test Network
- JavaScript Asset Transfer Chaincode
- Linux Terminal

Procedure

Step 1: Environment Configuration
Install and configure Docker, Ubuntu and the required dependencies. Download the Hyperledger Fabric samples, Docker images and command-line binaries.
Step 2: Fabric Installation Verification
Configure the Fabric executable path and verify the installation using:
peer version
Step 3: Network Initialization
Navigate to the Fabric test-network directory and execute:
./network.sh up createChannel -c mychannel
This starts two peer organizations, an ordering service and creates the communication channel.
Step 4: Network Verification
Execute docker ps to verify that the peer and orderer containers are running.
Step 5: Chaincode Deployment
Deploy the JavaScript asset management chaincode using:
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript
The chaincode is installed and approved by both organizations before being committed to the channel.
Step 6: Peer Configuration
Configure the required CORE_PEER_* environment variables to establish the identity, address and TLS certificates of Organization 1.
Step 7: Ledger Initialization
Invoke the InitLedger function to insert six predefined assets into the blockchain ledger.
Step 8: Asset Retrieval
Execute GetAllAssets to display the available assets and ReadAsset to retrieve the details of a selected asset.
Step 9: Asset Creation
Invoke CreateAsset to add a new asset with the following attributes:

- Asset ID: asset7
- Color: Green
- Size: 12

- Owner: Khushi
- Appraised Value: 950

Step 10: Ownership Transfer

Execute TransferAsset to change the owner of asset7 from Khushi to Tomoko. Verify the updated ownership using ReadAsset.

Step 11: Asset Existence Verification

Execute AssetExists for an asset ID that is not present in the ledger to verify the existence-checking functionality.

Step 12: Network Termination

Stop the test network using:
./network.sh down

Code & Output:

Step 1 — Set up WSL and Ubuntu

wsl --version | wsl -l -v | wsl.exe --install ubuntu

WSL 3.0.1.0 has no installed distributions, so Ubuntu is downloaded and installed.

```
C:\Users\heram>wsl --version
WSL version: 3.0.1.0
Kernel version: 6.18.40.1-1
WSLg version: 1.0.79
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.9457

C:\Users\heram>wsl -l -v
Windows Subsystem for Linux has no installed distributions.
You can resolve this by installing a distribution with the instructions
below:

Use 'wsl.exe --list --online' to list available distributions
and 'wsl.exe --install <Distro>' to install.

C:\Users\heram>wsl.exe --install ubuntu
Downloading: Ubuntu
[==============            24.3%                           ]
```

Step 2 — Create the default Unix user

The name “Khushi” is rejected because a valid username must start with a lowercase letter, so “khushi” is created and its password is set.

```
Create a default Unix user account: Khushi
Invalid username. A valid username must start with a lowercase letter or
 underscore, and can contain lowercase letters, digits, underscores, and
 dashes.
Create a default Unix user account: khushi
New password:
Retype new password:
passwd: password updated successfully
usermod: no changes
Help improve Ubuntu!
```

Step 3 — Fix Docker permissions

sudo usermod -aG docker $USER | sudo apt install util-linux-extra

docker ps first fails with “permission denied”. The user is added to the docker group and util-linux-extra (which provides newgrp) is installed.

```
ved@LAPTOP-ENAG2HIV:~$ docker ps
permission denied while trying to connect to the docker API at unix:///v
ar/run/docker.sock
ved@LAPTOP-ENAG2HIV:~$ sudo usermod -aG docker $USER
[sudo: authenticate] Password:
ved@LAPTOP-ENAG2HIV:~$ newgrp docker
Command 'newgrp' not found, but can be installed with:
sudo apt install util-linux-extra
ved@LAPTOP-ENAG2HIV:~$ sudo apt install util-linux-extra
Installing:
  util-linux-extra

Installing dependencies:
  liblastlog2-2

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 373 kB
  Space needed: 1405 kB / 1025 GB available

Continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 liblastlog2-2
 amd64 2.41.3-3ubuntu2 [26.7 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute/main amd64 util-linux-ex
tra amd64 2.41.3-3ubuntu2 [346 kB]
Fetched 373 kB in 2s (200 kB/s)
Selecting previously unselected package liblastlog2-2:amd64.
(Reading database ... 35908 files and directories currently installed.)
Preparing to unpack .../liblastlog2-2_2.41.3-3ubuntu2_amd64.deb ...
Unpacking liblastlog2-2:amd64 (2.41.3-3ubuntu2) ...
Selecting previously unselected package util-linux-extra.
Preparing to unpack .../util-linux-extra_2.41.3-3ubuntu2_amd64.deb ...
Unpacking util-linux-extra (2.41.3-3ubuntu2) ...
Setting up liblastlog2-2:amd64 (2.41.3-3ubuntu2) ...
Setting up util-linux-extra (2.41.3-3ubuntu2) ...
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2.3) ...
ved@LAPTOP-ENAG2HIV:~$
```

Step 4 — Install dependencies

sudo apt install jq curl git -y

jq is installed; curl and git were already at the newest version.

```
ved@LAPTOP-ENAG2HIV:~$ sudo apt install jq curl git -y
curl is already the newest version (8.18.0-1ubuntu2.4).
curl set to manually installed.
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
Installing:
  jq

Installing dependencies:
  libjq1  libonig5

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
  Download size: 393 kB
  Space needed: 1179 kB / 1025 GB available
```

Step 5 — Install Node.js

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

The NodeSource repository is added.

```
ved@LAPTOP-ENAG2HIV:~$ curl -fsSL https://deb.nodesource.com/setup_20.x
| sudo -E bash -
sudo: preserving the entire environment is not supported, '-E' is ignore
d
2026-10-08 03:47:20 - Installing pre-requisites
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137
 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 k
B]
Get:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137
 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Pac
kages [598 kB]
```

sudo apt install nodejs -y

```
ved@LAPTOP-ENAG2HIV:~$ sudo apt install nodejs -y
Installing:
  nodejs

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 93
  Download size: 32.2 MB
  Space needed: 196 MB / 1024 GB available
```

Step 6 — Clone fabric-samples

git clone https://github.com/hyperledger/fabric-samples

```
ved@LAPTOP-ENAG2HIV:~$ git clone https://github.com/hyperledger/fabric-s
amples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 16058, done.
remote: Counting objects: 100% (2970/2970), done.
remote: Compressing objects: 100% (567/567), done.
remote: Total 16058 (delta 2593), reused 2403 (delta 2403), pack-reused
13088 (from 2)
Receiving objects: 100% (16058/16058), 24.09 MiB | 1.25 MiB/s, done.
Resolving deltas: 100% (9241/9241), done.
ved@LAPTOP-ENAG2HIV:~/fabric-samples$
```

Step 7 — Download the Fabric binaries and Docker images

./install-fabric.sh docker samples binary

The Fabric v2.5.16 binaries are downloaded and then the Docker images for orderer, ccenv, baseos and CA are pulled.

```
ved@LAPTOP-ENAG2HIV:~/fabric-samples$ curl -sSLO https://raw.githubuserc
ontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-sample
s main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.16 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/downlo
ad/v2.5.16/hyperledger-fabric-linux-amd64-2.5.16.tar.gz
===> Will unpack to: /home/ved/fabric-samples
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   C
urrent
                                 Dload  Upload  Total   Spent   Left   S
peed
  0      0   0      0   0      0      0      0
  0      0   0      0   0      0      0      0
  0      0   0      0   0      0      0      0
    0
  0      0   0      0   0      0      0      0
  1 121.8M   1  2.13M   0      0  1.37M      0   01:28   00:01   01:27
  3 121.8M   3  4.27M   0      0  1.66M      0   01:13   00:02   01:11
  5 121.8M   5  6.10M   0      0  1.69M      0   01:11   00:03   01:08
  5 121.8M   5  7.26M   0      0  1.57M      0   01:17   00:04   01:13
  6 121.8M   6  7.82M   0      0  1.38M      0   01:27   00:05   01:22
  6 121.8M   6  8.13M   0      0  1.21M      0   01:40   00:06   01:34
  7 121.8M   7  9.18M   0      0  1.19M      0   01:42   00:07   01:35 9
  8 121.8M   8 10.06M   0      0  1.15M      0   01:45   00:08   01:37 7
  8 121.8M   8 10.95M   0      0  1.12M      0   01:48   00:09   01:39 7
```

```
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-orderer:2.
5.16
ghcr.io/hyperledger/fabric-orderer:2.5.16
====>  ghcr.io/hyperledger/fabric-ccenv:2.5.16
2.5.16: Pulling from hyperledger/fabric-ccenv
91094e494159: Pull complete
1a3ecb2ae779: Pull complete
385fa033ccf1: Pull complete
d997db9b0914: Pull complete
c32d94060c0f: Pull complete
1f467a066682: Pull complete
d838ef601b3c: Download complete
Digest: sha256:807493f095f65140ad4cd4543a824c9021a848c8c7aace9bec512e854
5e870f1
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ccenv:2.5.
16
ghcr.io/hyperledger/fabric-ccenv:2.5.16
====>  ghcr.io/hyperledger/fabric-baseos:2.5.16
2.5.16: Pulling from hyperledger/fabric-baseos
9ba18f3569df: Pull complete
a151ebea138c: Pull complete
b52a0845a0f4: Pull complete
cbe99fed15ad: Download complete
Digest: sha256:109b4a15b28235db0dcf6ecaa7ffd41dbeeecee6551b231d8178dcf57
4a9157b
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-baseos:2.5
.16
ghcr.io/hyperledger/fabric-baseos:2.5.16
===> Pulling fabric ca Image
====>  ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
18afb8c8d2a4: Downloading  16.78MB/43.49MB
b9649d3f054a: Downloading  18.87MB/49.19MB
0b966e606d9a: Download complete
6f4ebca3e823: Downloading  17.83MB/29.54MB
c36bacd85367: Download complete
```

Step 8 — Verify the Docker images

docker images | grep fabric

The baseos, ca, ccenv, orderer and peer images are present.

```
      66.8MB
ved@LAPTOP-ENAG2HIV:~/fabric-samples$ docker images | grep fabric
ghcr.io/hyperledger/fabric-baseos:2.5.16    109b4a15b282        257MB
      83.2MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB
       123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.16     807493f095f6       1.01GB
       257MB
ghcr.io/hyperledger/fabric-orderer:2.5.16   1d918fc7e1e0        182MB
      50.7MB
ghcr.io/hyperledger/fabric-peer:2.5.16      58979b65744b        232MB
      66.8MB
hyperledger/fabric-baseos:2.5               109b4a15b282        257MB
      83.2MB
```

Step 9 — Start the test network and create the channel

./network.sh up createChannel

Certificates for Org1, Org2 and the order are generated and the containers (peer0.org1, peer0.org2 and order) are started with the channel “mychannel”.

```
ved@LAPTOP-ENAG2HIV:~/fabric-samples$ cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and C
LI delay of '3' seconds and using database 'leveldb with crypto from 'cr
yptogen'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
/home/ved/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-or
g1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-or
g2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-or
derer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
 ✔Network fabric_test                  Created                    0.1s
 ✔Volume compose_peer0.org2.example.com Created                   0.0s
 ✔Volume compose_orderer.example.com    Created                   0.0s
 ✔Volume compose_peer0.org1.example.com Created                   0.0s
 ✔Container peer0.org2.example.com      Started                   0.8s
 ✔Container peer0.org1.example.com      Started                   0.6s
 ✔Container orderer.example.com         Started                   1.1s
CONTAINER ID   IMAGE                                  COMMAND             C
REATED         STATUS                 PORTS
```

Step 10 — Deploy the chaincode

./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript

The “basic” chaincode is packaged and installed on the peers of Org1 and Org2.

```
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ ./network.sh deployCC
 -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javas
cript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer
-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15c
bcc7f59b629$'
+ peer lifecycle chaincode queryinstalled --output json
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

Step 11 — Chaincode committed on the channel

The chaincode definition (version 1.0, sequence 1) is approved by Org1MSP and Org2MSP and committed on “mychannel”.

```
+ res=0
2026-10-08 04:14:01.323 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid
[b8dc37b58e486ac71c33739abf2a18af7c241b59e4b58dfd61708df1344858b6] commi
tted with status (VALID) at localhost:7051
2026-10-08 04:14:01.323 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid
[b8dc37b58e486ac71c33739abf2a18af7c241b59e4b58dfd61708df1344858b6] commi
tted with status (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 second
s.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name b
asic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychann
el':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin:
vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychanne
l'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 second
s.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name b
asic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychann
el':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin:
vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychanne
l'
Chaincode initialization is not required
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$
```

Step 12 — Set the peer environment and initialize the ledger

peer chaincode invoke ... -c '{"function":"InitLedger","Args":[]}'

The environment variables are set for Org1 and InitLedger is invoked, which returns status:200.

```
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ export FABRIC_CFG_PAT
H=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/org
anizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.c
om/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organiz
ations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/m
sp
export CORE_PEER_ADDRESS=localhost:7051
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode invoke
 -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tl
s --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerO
rganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --
peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/tes
t-network/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt -c '{"function":"InitLedger","Args":[]}'
2026-10-08 04:16:42.921 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$
```

Step 13 — Query the assets

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

GetAllAssets lists asset1 to asset6, and ReadAsset returns the details of asset5 (owner Adriana).

```
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode query
-C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Si
ze":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset
2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Col
or":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}
,{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Siz
e":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"ass
et5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":80
0,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"as
set"}]
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode query
-C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","S
ize":15,"docType":"asset"}
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$
```

Step 14 — Create and transfer an asset

CreateAsset asset7, blue, 50, Manav, 100 | TransferAsset asset7, Alice

asset7 is created with owner Manav and then transferred to Alice; both invocations return status:200.

```
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode invoke
 -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tl
s --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerO
rganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --
peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/tes
t-network/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt -c '{"function":"CreateAsset","Args":["asset7
","blue","50","Manav","100"]}'
2026-10-08 04:19:43.048 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":
\"asset7\",\"Color\":\"blue\",\"Size\":50,\"Owner\":\"Manav\",\"Appraise
dValue\":100}"
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode invoke
 -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tl
s --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerO
rganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --
peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/tes
t-network/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt -c '{"function":"TransferAsset","Args":["asse
t7","Alice"]}'
2026-10-08 04:19:53.640 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200 payload:"Manav"
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$
```

Step 15 — Bring the network down

./network.sh down

The containers, volumes and the fabric_test network are removed.

```
2026-10-08 04:21:22.137 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200 payload:"Alice"
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ cd ~/fabric-samples/t
est-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔Container peer0.org2.example.com      Removed                    1.0s
 ✔Container peer0.org1.example.com      Removed                    0.9s
 ✔Container orderer.example.com         Removed                    0.4s
 ✔Volume compose_orderer.example.com    Removed                    0.0s
 ✔Volume compose_peer0.org2.example.com Removed                    0.0s
 ✔Network fabric_test                   Removed                    0.2s
 ✔Volume compose_peer0.org1.example.com Removed                    0.0s
```

Commands Used:

Install dependencies:

```
sudo apt update && sudo apt upgrade -y
sudo apt install jq curl git -y
curl -fsSL https://deb.nodesource.com/setup_20.x
sudo -E bash -
sudo apt install nodejs -y
```

Clone and install Fabric:

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
docker images
grep fabric
```

Path and network:

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

Deploy chaincode:

```
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
-ccl javascript
```

Set peer environment:

```
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrgani
zations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizati
ons/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Initialize, query, create, transfer:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile <orderer tlsca cert> -C mychannel -n basic
--peerAddresses localhost:7051 --tlsRootCertFiles <org1 ca.crt> --peerAddresses
localhost:9051 --tlsRootCertFiles <org2 ca.crt> -c
'{"function":"InitLedger","Args":[]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
(invoke) -c '{"function":"CreateAsset","Args":["asset7","blue","50","Manav","100"]}'
(invoke) -c '{"function":"TransferAsset","Args":["asset7","Alice"]}'
```

Observations and Inferences

1. The Hyperledger Fabric test network consists of two peer organizations and one ordering service connected through mychannel.
2. Docker containers provide an isolated environment for running blockchain components.
3. Chaincode deployment requires approval from the participating organizations.
4. The InitLedger function populates the ledger with six sample assets.
5. The GetAllAssets and ReadAsset functions retrieve asset information without modifying the ledger.
6. The CreateAsset function adds asset7 with the specified attributes.

7. The TransferAsset function updates the owner from Khushi to Tomoko while retaining the other asset properties.
8. The AssetExists function returns false when the requested asset is unavailable.
9. Write transactions require endorsement and commitment, whereas query operations retrieve existing ledger information.

Result

The experiment demonstrates the creation of a Hyperledger Fabric network, deployment of asset management chaincode and execution of asset operations such as initialization, retrieval, creation, ownership transfer and existence verification.

Conclusion

Hyperledger Fabric provides a secure and controlled environment for managing blockchain transactions among authorized organizations. This experiment demonstrates how chaincode automates asset-related operations and how distributed peers maintain consistent ledger records. It also highlights the importance of transaction endorsement, ordering and validation in a permissioned blockchain system.